

Programmierpraktikum

MacMahonMosaik

Anton Burmester
Wirtschaftsinformatik

Inhaltsverzeichnis

Benutzerhandbuch	4
1. Ablaufbedingungen	4
2. Programminstallation/Programmstart	4
3. Bedienungsanleitung	5
3.1. Spielablauf und Regeln:	5
Nicht korrekt gelöstes Spielfeld korrekt gelöstes Spielfeld	6
3.2. Aufbau der Oberfläche:	6
3.3. Spielbedienung:	9
4. Fehlermeldungen	10
4.1 Fehlermeldungen	10
4.2. Nachrichten	11
5. Spielstandsdatei Struktur	11
Programmiererhandbuch	12
1. Entwicklungskonfiguration	12
1.1 macOS (Entwicklungsumgebung und Testumgebung)	12
1.2 Windows (Build Umgebung)	12
1.3 Projekt	12
2. Problemanalyse und Realisation	13
2.1. Speicherort der Tiles (Spielsteinauswahl) Klasse	13
2.2. Bewegen der Spielsteine und Lochsteine	13
2.3. Speicherung der Lochsteine als Tiles Nutzlast in Game oder GameField	15
2.4. Tiles Klasse Umsetzung	15
2.5. Speichern der freien Felder eines Spielfelds für den Algorithmus zum Lösen des Spielfelds	17
3. Algorithmen	18
3.1 Algorithmus zum Lösen des Spielfelds	18
4. Programmorganisationsplan	19
4.1 Graphischer Programmorganisationsplan	19
4.2 Programmorganisationsplan Erklärung	20

5. Dateien	21
6. Programmtests	21
7. Bekannte Fehler des Programms:	23

Benutzerhandbuch

1. Ablaufbedingungen

Systemanforderungen für das Programm:

Betriebssystem:

Windows 11

MacOS Sequoia 15.6

Java JDK:

Temurin 21 oder neuer

2. Programminstallation/Programmstart

Dateiname des Programms: pp_MacMahonMosaik_Burmester-1.0-SNAPSHOT.jar

Windows:

1. Windows Aufgabeaufforderung (CMD) starten
2. in das Verzeichnis navigieren in welchem dieses Programm als .jar liegt
3. Programm starten mithilfe des Befehls:
„java -jar pp_MacMahonMosaik_Burmester-1.0-SNAPSHOT.jar“

macOS:

1. Terminal starten
2. in das Verzeichnis navigieren in welchem dieses Programm als .jar liegt
3. Programm starten mithilfe des Befehls:
„java -jar pp_MacMahonMosaik_Burmester-1.0-SNAPSHOT.jar“

3. Bedienungsanleitung

Bei diesem Programm handelt es sich um das Spiel „MacMahonMosaik“.

Ziel des Spiels ist es, ein Mosaikfeld so zu lösen, dass alle Felder mit Spielsteinen belegt sind und deren Farbflächen mit den ihnen jeweils angrenzenden Farbflächen auf Spielfeld und Randflächen farblich übereinstimmen.



3.1. Spielablauf und Regeln:

Es gibt maximal 24 farbige Spielsteine. Jeder Spielstein besteht aus 4 diagonal geteilten Feldern. Die Farbflächen der Felder sind rot, gelb oder grün. Die vier Felder eines Spielsteins können aus 4 gleichen Farben bestehen oder aus 3 verschiedenen Farben oder aus 2 verschiedenen Farben. Zu Beginn eines Spiels muss der Spieler das Spiel zunächst initialisieren. Hierfür muss er die gewünschte Anzahl der Spielfelder auswählen. Das Spiel kann mit 2 x 2 bis 6 x 6 Feldern gespielt werden. Falls es mehr als 24 freie Felder gibt, wird die Differenz automatisch mit Lochsteinen gefüllt. Lochsteine im Spielfeld werden zunächst automatisch positioniert, der Spieler kann die Position der Lochsteine aber auch selbst verändern. Zudem muss der Spieler die Farben der Randfelder festlegen.

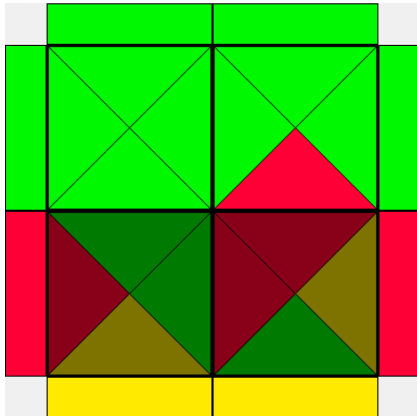
Ist dies erledigt, befindet sich der Spieler jetzt vor einem leeren Spielfeld mit mit 2 x 2 bis 6 x 6 Feldern, verschiedenfarbigen Randfeldern, gegebenenfalls platzierten Lochsteinen und neben dem Spielfeld platzierten, verschiedenfarbigen Spielsteinen, die das Spiel automatisch zu Auswahl zur Verfügung stellt.

Um das Spiel korrekt zu lösen, muss der Spieler das Spielfeld mit farblich zu allen Seiten passenden Spielsteinen füllen. Ein Spielstein passt nur dann korrekt, wenn alle seine Farbflächen mit der ihm jeweils angrenzenden Farbfläche übereinstimmt. Eine Ausnahme bilden die Lochsteine, da diese alle Farben akzeptieren.

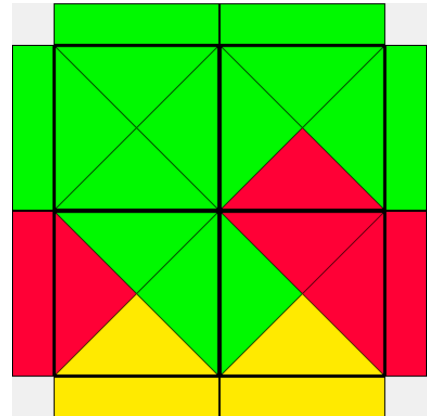
Der Spieler hat das Spiel korrekt gelöst, wenn er alle freien Spielfeldfelder mit Spielsteinen gefüllt hat und diese farblich mit den ihnen jeweils angrenzenden Farbflächen übereinstimmen.

Beispiel für ein korrekt gelöstes und ein nicht korrekt gelöstes Spielfeld:

Bei dem korrekt gelösten Spielfeld stimmen alle aneinander grenzenden Farbränder der gelegten Spielsteine farblich mit ihren Nachbarn und den angrenzenden gefärbten Randflächen überein. Bei dem nicht korrekt gelösten Spielfeld stimmen Ränder der Spielsteine bezüglich ihrer Farbe teils nicht miteinander überein.



Nicht korrekt gelöstes Spielfeld



korrekt gelöstes Spielfeld

3.2. Aufbau der Oberfläche:

Das Programm hat oben ein Auswahlménü (1)
Links mittig befindet sich das Spielfeld (2)
Rechts befindet sich die Spielsteinauswahl (3)
In der Spielsteinauswahl befinden sich die Spielsteine (4.1)
Auf dem Spielfeld befinden sich, falls notwendig die Lochsteine (4.1)
Editor (auf der Abbildung links nicht zu sehen) (5)



1. Das Auswahlm Menü:



1. Spiel



1.1. Spiel neu starten

Startet ein aktives Spiel neu, indem alle Spielsteine vom Spielfeld genommen werden und wieder der Spielsteinauswahl zugeführt werden. Lochsteine und Randfärbung bleiben bestehen. Sollte ein Spiel beendet worden sein, oder allgemein nicht aktiv sein, kann es hiermit gestartet werden.

1.2. Spielstand laden

Öffnet den Dateimanager des jeweiligen Betriebssystems mithilfe der Spieler dann einen Spielstand laden kann.

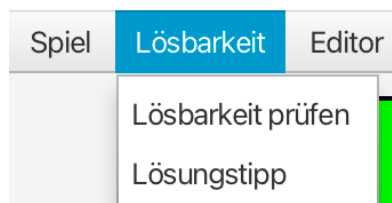
1.3. Spielstand speichern

Öffnet den Dateimanager des jeweiligen Betriebssystems mithilfe der Spieler dann einen Spielstand speichern kann.

1.4. Spiel beenden

Beendet ein aktuelles Spiel.

2. Lösbarkeit



2.1 Lösbarkeit prüfen

Prüft bei Spielfeldern, welche maximal 18 innere Felder haben, ob diese mit den verfügbaren Spielsteinen lösbar sind.

2.2. Lösungstipp

Legt bei Spielfeldern, welche maximal 18 innere Felder haben, einen passenden Spielstein in das Spielfeld. Der Spieler kann das Spiel weiter spielen.

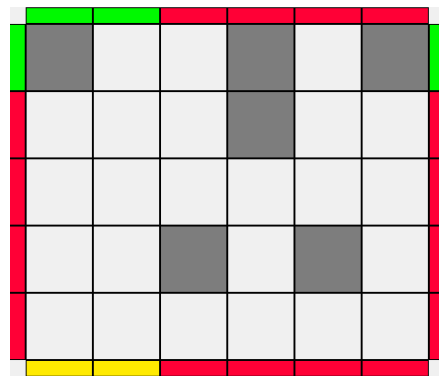
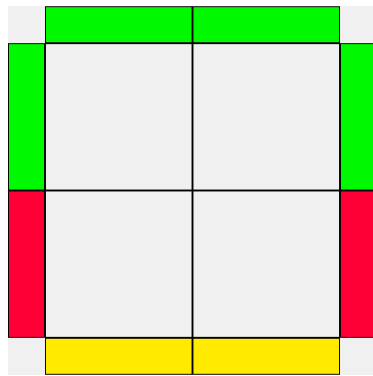
3. Editor



3.1. Bearbeitungsmodus

Startet einen Modus, in welchem der Spieler die Anzahl der Felder auswählen kann, die Farben am Rand des Spielfelds auswählen kann, sowie, falls mehr als 24 innere Spielfelder ausgewählt wurden, die vom Spiel automatisch gesetzten Lochsteine in der Position im Spielfeld verändern kann.

2. Spielfeld:

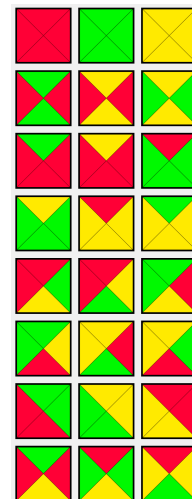


Das Spielfeld besteht aus inneren Feldern, welche von Randfeldern umgeben sind. Der innere Teil des Spielfelds besteht aus mindestens 2x2 Feldern bis maximal 6x6 Feldern. Also minimal 4 innere Felder und maximal 36. Das innere Spielfeld wird durch Randfelder umrandet. Die Ecken sind nicht ausgefüllt. Das Spielfeld kann quadratisch oder auch rechteckig sein.

Die Farben der Randfelder kann der Spieler beliebig festlegen. Hierbei kann er zwischen den Farben rot, grün, und gelb wählen.

3. Spielsteinauswahl-Feld:

Fläche neben dem Spielfeld, welche die noch nicht platzierten Spielsteine enthält.

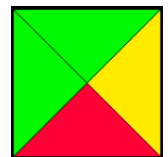


4. Steine:

4.1 Spielsteine:

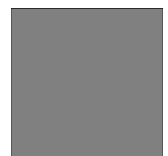
24 individuelle Spielsteine, mit welchen das innere Spielfeld gefüllt werden soll.

Diese sind durch zwei Diagonalen in 4 Bereiche eingeteilt. Diese 4 Bereiche sind in den Farben rot, grün oder gelb eingefärbt.



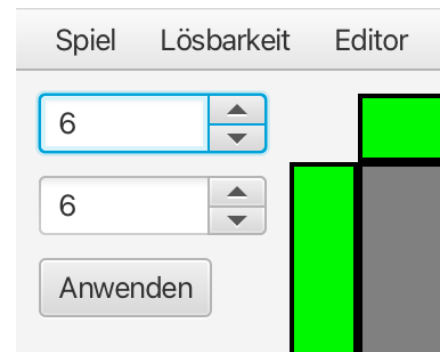
4.2 Lochsteine:

Zusätzlich zu den Spielsteinen, gibt es Lochsteine welche ab einem 5x5 großen Spielfeld im Spielfeld automatisch gesetzt werden oder vom Spieler positioniert werden können, da es ab dieser Spielfeldgröße mindestens 25 freie Felder gibt, jedoch nur 24 Spielsteine. Bei einem Lochstein ist es egal welche Spielsteine an diesen angrenzen, da er jede Randfarbe akzeptiert.



5. Editor-Mode:

Im Editor Mode kann der Spieler die Größe des Spielfelds bestimmen, den Farbe der Randfelder auswählen sowie, falls vorhanden, Lochsteine im Spielfeld umplatzieren.



3.3. Spielbedienung:

Das Spiel ist sehr intuitiv bedienbar.

1. Menü:

Durch Anklicken eines Menüabschnitts diesen öffnen. Durch Anklicken des Unterpunkts des Menüabschnitts, die Aktion ausführen.

2. Spielfeld:

Editor-Mode aktiviert:

Randfarbe ändern: durch Anklicken von Randfeldern

Lochsteine im Spielfeld bewegen: durch ziehen und loslassen von Lochsteinen

Editor-Mode deaktiviert:

Spielstein aus Spielsteinauswahl ins Spielfeld: gewünschten Spielstein aus Spielsteinauswahl ziehen und über dem Ziel Feld im Spielfeld positionieren und loslassen.

Spielstein von Spielfeld-Feld zu Spielfeld-Feld: Spielstein der bewegt werden soll ziehen und über dem Ziel Feld positionieren und loslassen.

Spielstein von Spielfeld zurück in Spielsteinauswahl: Spielstein der wieder zurück in die Spielsteinauswahl soll, ziehen und über der Spielsteinauswahl positionieren und loslassen.

3. Spielsteinauswahl:

Editor-Mode deaktiviert:

Spielstein aus Spielsteinauswahl ins Spielfeld: gewünschten Spielstein aus Spielsteinauswahl ziehen und über dem Ziel Feld im Spielfeld positionieren und loslassen.

Spielstein von Spielfeld zurück in Spielsteinauswahl: Spielstein der wieder zurück in die Spielsteinauswahl soll, ziehen und über der Spielsteinauswahl positionieren und loslassen.

4. Steine:

4.1:

Spielsteine können durch direktes Anklicken mit der rechten Maustaste gedreht werden. In der Spielsteinauswahl und im Spielfeld.

4.2:

Ein Drehen der Lochsteine hat keine Auswirkungen.

5. Editor-Mode:

Die Größe des Spielfelds kann durch Anklicken der Breitenauswahl und Höhenauswahl ausgewählt werden. Das Setzen der Randfarbe und bewegen der Lochsteine ist im Punkt „2. Spielfeld“ beschrieben.

4. Fehlermeldungen

4.1 Fehlermeldungen

Code	Ursache	Lösung
101	Fenster kann aufgrund von Sicherheitseinstellungen nicht geöffnet werden.	Sicherstellen, dass das Programm berechtigt ist, neue Fenster zu öffnen.
201	Es wurde keine Datei beim „Spielstand speichern“ oder „Spielstand Laden“ erfolgreich ausgewählt.	Datei erfolgreich auswählen um diese zu laden oder beim Speichern eines Spielstands diese korrekt benennen. Das Fenster des Dateimanagers schließt sich, sobald dies geschehen ist.
202	Die angeforderte Spielstandsdatei konnte nicht eingelesen werden.	Überprüfen, dass die Datei nicht beschädigt ist, nicht von einem anderen Prozess verarbeitet wird. Ansonsten in einen Ordner mit Schreib und Leserechten verschieben.
302	Die angeforderte Spielstandsdatei hat .json Format, enthält jedoch keinen Schlüssel namens „field“	Den Schlüssel des Felds in der .json Datei umbenennen zu „field“.
303	Die angeforderte Spielstandsdatei enthält zwar einen Schlüssel namens „field“, jedoch speichert dieser kein JSONArray.	Bei dem Schlüssel „field“ in der .json Datei das Spielfeld in zweidimensionalem Array Format speichern.
304	Das Spielfeld der Spielstandsdatei ist entweder zu groß oder zu klein.	Die Größe des Spielfelds in der .json Datei anpassen, so dass dieses mit Rand nicht größer als 8x8 und nicht kleiner als 4x4 ist.
305	Die angeforderte Spielstandsdatei ist leer.	Die Datei mit einem Spielfeld füllen
401	Mindestens einer der Spielsteine in der Spielstandsdatei ist falsch benannt.	Spielsteine in der Datei korrekt benennen.
402	Mindestens ein Stein auf dem Rand des Spielfelds, ist nicht mit dem Rand kompatibel.	Auf dem Rand nur Randkompatible Spielsteine platzieren.
403	Mindestens auf einer Ecke liegt ein nicht Eckkompatibler Stein.	Auf den Ecken nur Eckkompatible Steine platzieren.
404	Mindestens ein Spielstein vom selben Muster ist mehrmals auf dem Spielfeld platziert.	Im mittleren Spielfeld jeden Spielstein nur einmal platzieren. Derselbe Spielstein darf auch nicht mehrmals unterschiedlich rotiert vorkommen.
405	Im mittleren Spielfeld sind entweder zu wenig Lochsteine oder zu viele platziert.	Soviel Lochsteine wie notwendig platzieren.
503	Mindestens ein Feld des Randes ist nicht farbig.	Im Editor-Mode dieses Feld in der gewünschten Farbe färben.
504	Die aktuell angeforderte Funktion kann nicht ausgeführt werden, da der Editor-Mode aktiv ist.	Editor-Mode über das Menü deaktivieren.

4.2. Nachrichten

Sehen aus wie Fehler, haben aber keinen Fehler als Auslöser.

Code	Bedeutung
0	Das Spielfeld wurde erfolgreich gelöst und ist somit zu Ende
10	Das Spielfeld ist lösbar
11	Das Spielfeld ist nicht lösbar
21	Kein Tipp kann gelegt werden, da das Spielfeld nicht lösbar ist und es somit keine Tipps gibt.
31	Da es mehr als 18 freie Spielfeld Felder gibt, wird die Lösbarkeitsprüfung sowie das Tipp legen übersprungen

5. Spielstandsdatei Struktur

```
{"field":  
  ["NNNN","NNGN","NNGN","NNRN","NNRN","NNRN","NNNN"],  
  ["NGNN","NNNN","NNNN","NNNN","NNNN","NNNN","NNNG"],  
  ["NRNN","NNNN","RRRR","NNNN","NNNN","NNNN","NNNR"],  
  ["NRNN","NNNN","NNNN","NNNN","RYGR","NNNN","NNNR"],  
  ["NRNN","NNNN","NNNN","NNNN","NNNN","NNNN","NNNR"],  
  ["NNNN","YNNN","YNNN","RNNN","RNNN","RNNN","NNNN"]  
}]
```

Schlüssel: „field“

Größe: 4x4 - 8x8 (mit Rand)

Erlaubte Ecksteine: „NNNN“

Erlaubte Randsteine: „GGGG“, „YYYY“, „RRRR“

Erlaubte Spielsteine: „RRRR“, „GGGG“, „YYYY“, „GRGR“, „YRYR“, „YGYG“, „GRRR“, „YRRR“, „RGGG“, „YGGG“, „RYYY“, „GYYY“, „RGYR“, „RYGR“, „GRYG“, „GYRG“, „YRGY“, „YGRY“, „GGRR“, „YYGG“, „RRYY“, „GRYR“, „RGYR“, „RYGY“ (falls es mehr als 24 freie Felder gibt noch „NNNN“ für die Differenz nach 24 gelegten Spielsteine freien Felder)

Programmiererhandbuch

1. Entwicklungskonfiguration

1.1 macOS (Entwicklungsumgebung und Testumgebung)

Betriebssystem	macOS Sequoia 15.6 (Build 24G84) (Architektur aarch64) (Apple Silicon)
IDE	IntelliJ IDEA 2024.1.4 (Community Edition) (Build #IC-241.18034.62)
IDE Runtime Version	17.0.11+1-b1207.24 aarch64
SDK	Eclipse Temurin 21.0.3
Projekt JDK	Irrelevant, da hier nicht die .jar erzeugt oder ausgeführt wird
Versionierung	git version 2.39.5 (Apple Git-154)

1.2 Windows (Build Umgebung)

Betriebssystem	Windows 11 24H2 (Build 26100, 4652) (Architektur amd64)
IDE	IntelliJ IDEA 2024.1.6 (Community Edition) (Build #IC-241.19072.14)
IDE Runtime Version	17.0.11+1-b1207.30 amd64
SDK	Eclipse Temurin 21.0.4
Projekt JDK	OpenJDK 21.0.4 (64 Bit) Runtime Environment Temurin-21.0.4+7 (Build 21.0.4+7-LTS) Server VM Temurin-21.0.4+7 (Build 21.0.4+7, mixed mode, sharing)
Versionierung	git version 2.46.1.windows.1

1.3 Projekt

Projekt SDK	Eclipse Temurin 21.0.3
Build Tool	Apache Maven 3.9.9
JavaFX SDK	21.0.3 (Module: java-fxml und javafx-control)
JavaFX Maven Plugin	0.0.8
Maven Compiler Plugin	3.11.0
Maven Surefire Plugin	3.1.2
Maven Jar Plugin	3.3.0
Maven Shade Plugin	3.5.1
Libaries	Gson 2.10.1, JUnit 5.10.0

2. Problemanalyse und Realisation

2.1. Speicherort der Tiles (Spielsteinauswahl) Klasse

Problemanalyse:

Für die Umsetzung wird eine Klasse benötigt, die alle nicht platzierten Spielsteine speichert.

Realisationsanalyse:

1. speichern der Nutzlast der Spielsteinauswahl in der Game Klasse

Die Game Klasse ist die Schnittstelle zwischen gui und logik.

Sie enthält eine Nutzlast der Spielfeld Klasse und hier zusätzlich eine der Spielsteinauswahl.

Hierdurch muss sie bei Zügen zwischen dem Spielfeld und der Spielsteinauswahl den Zug zwischen beiden koordinieren.

Dies ist prinzipiell möglich, jedoch soll ein Spielfeld auch klonbar sein.

Hier müsste sich die Spielfeld Klasse dann um das Entfernen der gelegten Spielsteine einer neuen Spielsteinauswahl Instanz kümmern.

Vorteil: keiner

Nachteil: unübersichtlich (teils kümmert sich die Game Klasse um das Entfernen von Spielsteinen aus der Spielsteinauswahl und teils die GameField Klasse)

2. Speichern der Nutzlast der Spielsteinauswahl in der GameField Klasse

Die GameField Klasse enthält nun das Spielfeld sowie die Nutzlast der Spielsteinauswahl.

Somit hat diese die volle Verantwortung über beide und kann alles ganz einfach koordinieren denn sie hat nun zwei Methoden.

Einmal eine, welche einen Spielstein in das Spielfeld legt und diesen aus der Spielsteinauswahl löscht falls er aus dieser kommt.

Und einmal eine, welche den Spielstein aus dem Spielfeld löscht und diesen der Spielsteinauswahl wieder hinzufügt.

Vorteil: alles strikt in der GameField Klasse koordiniert. Methoden, welche das Spielfeld und lösen können sich auf ihren Prozess konzentrieren und müssen sich nicht um Konsistenz zwischen Spielsteine und Spielsteinauswahl sorgen.

Nachteil: keiner

Realisationsbeschreibung:

Es wurde sich für Weg zwei entschieden.

Die Nutzlast der Spielsteinauswahl wird in der GameField Klasse gespeichert und gehandhabt.

Wird ein Spielstein gelegt, so gibt es die entsprechende Methode, welche den Spielstein im Spielfeld legt und gleichzeitig falls dieser aus der Spielsteinauswahl kommt löscht.

Wird ein Spielstein zurückgesetzt, so gibt es die entsprechende Methode, welche den Spielstein aus dem Spielfeld löscht und diesem der Spielsteinauswahl wieder hinzufügt.

2.2. Bewegen der Spielsteine und Lochsteine

Problemanalyse:

Laut Aufgabenstellung sollen Spielsteine zwischen der Spielsteinauswahl und dem Spielfeld sowie innerhalb des Spielfelds bewegbar sein. Außerdem sollen Lochsteine innerhalb des Spielfelds beweglich sein.

Hierbei ist freigestellt, ob diese Interaktion über Drag and Drop laufen soll oder über Anklicken des Startfelds und Anklicken des Zielfelds.

Realisationsanalyse:

Entweder kann die Bewegung über Drag and Drop laufen oder über Anklicken des Startfelds und Anklicken des Zielfelds.

Innerhalb der beiden Alternativen gibt es jedoch noch viel Spielraum bezüglich der Umsetzung.

1. Anklicken des Startfelds und Anklicken des Zielfelds:

Die Steine, welche bewegt werden sollen, bekommen Listener, welche Auslösen sobald diese angeklickt werden.

Diese Listener speichern global in einer Klasse, welcher Stein verschoben werden soll. Es wird immer der am neusten angeklickte Steing gespeichert.

Nun haben auch die Ziele Listener also die Platzhalter in den Zellen der Spielsteinauswahl sowie des Spielfelds.

Werden diese angeklickt, so rufen sie die entsprechende Game Methode auf und übergeben je nachdem, um welche es sich handelt, Start und oder Ziel und/oder SpielsteinID.

Die entsprechende Methode prüft dann, ob der Zug valide ist und führt gegebenenfalls Änderungen an der Grafik durch.

2. Drag and Drop mit Platzhaltern welche gleichzeitig Drag and Drop Listener haben:

Die Platzhalter der Zellen der Spielsteinauswahl sowie des Spielfelds haben Drag and Drop Listener.

Die Steine haben keine Drag Listener, da diese ja schon auf dem Platzhaltern vorhanden sind.

Wird ein Stein per Drag bewegt, so wird dies vom aktuellen Platzhalter durch der Stein bemerkt und der Stein wird zum Bewegen freigegeben.

Wird dieser Stein nun über ein Feld gezogen und losgelassen, ruft der Drag Teil des Platzhalters die entsprechende Methode der Game Klasse auf und gibt je nachdem um welche es sich handelt Start und oder Ziel und oder SpielsteinID weiter.

Die entsprechende Methode prüft dann, ob der Zug valide ist und führt gegebenenfalls Änderungen an der Grafik durch.

Vorteile: keine

Nachteile: die gezogenen Objekte haben keine Listener und die Platzhalter alle. Hierdurch kann es schnell unübersichtlich werden. Außerdem ist es wenn es mehrere unterschiedliche Typen an Objekten zum bewegen gibt sehr umständlich.

3. Drag and Drop mit Platzhaltern welche Drop Listener haben und Steinen welche Drag Listener haben:

Die Felder auf welchen Objekte abgelegt werden können sollen, bekommen Platzhalter mit Drop Listenern.

Die Objekte welche per Drag bewegt werden sollen, bekommen Drag Listener.

Wird nun ein Objekt per Ziehen bewegt und über einem Ziel losgelassen, so ruft der Drop Listener des Zielfelds die entsprechende Game Methode auf und übergibt je nachdem um welche Bewegung es geht, Start und oder Ziel und oder SpielsteinID.

Vorteile: gegenüber 2. viel sauberer

Nachteile: keine

Realisationsbeschreibung:

Es wurde sich für die zweite Drag and Drop Variante entschieden.

Hierbei werden die Spielsteinauswahl und das Spielfeld, welche beide vom Typ GridPane sind mit Platzhaltern vom Typ StackPane für jede Zelle gefüllt.

Diese Platzhalter haben wiederum Listener, welche bemerken, sobald sich ein valider Spielstein über diesen befindet. Wird ein Spielstein nun losgelassen, ruft der Listener eine bestimmte Methode der Game Klasse auf.

Hierbei gibt es drei Stück.

1. Spielsteinauswahl zu Spielfeld (benötigt lediglich die ID des Spielsteins sowie die Ziel Koordinaten)

2. Spielfeld zu Spielfeld (benötigt lediglich die ID des Spielsteins sowie die Ziel Koordinaten)

3. Spielfeld zu Spielsteinauswahl (benötigt lediglich die ID des Spielsteins)

Die Platzhalter des Spielfelds enthalten die 1. und die 2. Methode und die Platzhalter der Spielsteinauswahl die 3. Methode.

Die Platzhalter der Spielsteinauswahl unterscheiden bei einem Drop eines Objekts, welche Methode dem Drag entspricht durch schauen der Herkunft des Objekts.

Kommt dieses aus der Spielsteinauswahl, die die Methode 1 genutzt, sonst die Methode 2.

Nun benötigt man noch Objekte, welche man bewegen kann.

Hierbei gibt es zwei verschiedene Typen.

1. Spielsteine vom Typ ImageView

2. Lochsteine vom Typ StackPane

Die Spielsteine werden im Spielfeld sowie zwischen dem Spielfeld und der Spielsteinauswahl bewegt.

Die Lochsteine werden hingegen nur innerhalb des Spielfelds bewegt.

Beide bekommen Drag Listener welche Bewegungen der Objekte registrieren.

Da die Lochsteine eine Lochstein ID haben und die Spielsteine eine Spielstein ID haben, können die Drop Listener einfach unterscheiden wer wohin darf.

Alternativ kann man hier da beide beweglichen Objekte vom unterschiedlichen Typ sind durch den Typ unterscheiden oder einfach schauen, aus welchem Array diese kommen, weil beide in Arrays vorgehalten werden.

Wird nun ein Objekt per Drag bewegt und per Drop auf ein valides Ziel losgelassen, wird dies über den Drop Listener an die jeweilige Game Methode weitergegeben. Diese überprüft den Zug, führt ihn aus und zeigt ihn falls valide grafisch durch den GUI Connector an.

2.3. Speicherung der Lochsteine als Tiles Nutzlast in Game oder GameField

Problemanalyse:

Laut Aufgabenstellung soll es auch Spielfeldgrößen geben, welche mehr als 24 Felder haben und somit mehr freie Felder enthalten, als die maximale Anzahl von 24 Spielsteinen.

Um diese Felder zu füllen, gibt es sogenannte Lochsteine.

Diese gibt es maximal 12-mal, da die maximale Spielfeldgröße 6x6 beträgt (ohne Rand) was 36 Felder ergibt, es aber nur 24 Spielsteine gibt.

Die Frage ist nun, ob man diese 12 Lochsteine einmalig initialisiert und als Tiles Nutzlast in der GameField Klasse speichert oder ob man auf diese einfach verzichtet.

Realisationsanalyse:

1. 12 Lochsteine als Tiles Nutzlast der GameField Klasse:

Die Lochsteine könnten einfach als eigene Tiles Klasse gehandhabt werden.

Diese würde dann 12 Spielsteine enthalten und wäre einmalig zu erzeugen, genauso wie die Tiles Klasse, welche die Spielsteine enthält.

2. Lochsteine bei Bedarf neu erzeugen:

Die Lochsteine frei nach Bedarf neu erzeugen, sobald das Spielfeld erzeugt wurde.

Diese liegen initial im Spielfeld und können unter bestimmten Voraussetzungen vom Nutzer bewegt werden.

Realisationsbeschreibung:

Es wurde sich für die zweite Alternative entschieden, da die Spielsteine alle zu jedem Zeitpunkt identische Merkmale aufweisen.

Wird ein Spielfeld erzeugt, welches mehr als 24 Felder enthält, so wird die Differenzanzahl zu den maximal vorhandenen 24 Spielsteinen an Lochsteinen erzeugt und im Spielfeld ab Position (1,1) platziert.

Wird ein Spielfeld geladen, werden die Lochsteine bei Feldern mit mehr als 24 Spielsteinen an den vorgegebenen Positionen platziert.

Diese Lösung hat keinen Nachteil gegenüber der Alternative und spart Verwaltungsaufwand.

2.4. Tiles Klasse Umsetzung

Problemanalyse:

Das Spiel bedarf mehreren Spielsteinen.

Diese müssen irgendwie gespeichert und verwaltet werden.

Realisationsanalyse:

1. Spielsteine (Tile) ununterbrochen in der Tiles (Spielsteinauswahl) Klasse halten:

Die Spielsteinklasse hat ein einmalig initialisiertes statisches Array dessen Größe der Anzahl an Spielsteinen entspricht.

Wird ein Spielstein im Spielfeld abgelegt, so bleibt dieser trotzdem trotzdem auch in der Tiles Klasse.

Augenscheinlicher Vorteil dieses Weges ist, dass die Reihenfolge der Spielsteine zu jedem Zeitpunkt identisch bleibt. Dies kann aber auch wie gleich in der dritten Alternative anders gelöst werden.

Der Nachteil hier ist Sauberkeit und Konsistenz, da der Spielstein an zwei Orten gleichzeitig liegt, was dem Spieler zwar nicht angezeigt wird, jedoch im Code zu Problemen führen kann.

2. Spielsteine (Tile) wenn gelegt aus der Spielsteinauswahl (Tiles) Klasse entfernt, jedoch die Position mit null ersetzt:

Sollten diese Spielsteine auf das Spielfeld gelegt werden, so wird ihre Position mit null als Platzhalter ersetzt, damit die Reihenfolge intakt bleibt.

Wird ein Spielstein zurück in die Spielsteinauswahl gelegt, so wird der Ordinale Wert des Spielsteingesichts abgerufen und der Spielstein an dem Wert wieder ins Array gesetzt, also der Platzhalter ersetzt mit dem richtigen Spielstein.

Da die Anzahl des Arrays hier kontinuierlich identisch bleibt, kann man auch hier mit einem statischen Array arbeiten.

Vorteil: ein Spielstein liegt entweder in der Spielsteinauswahl oder im Spielfeld (Konsistenz) und die Reihenfolge der Spielsteine bleibt intakt.

Nachteil: es wird mit Platzhaltern gearbeitet

3. Tile (Spielstein) wenn gelegt aus der Klasse entfernen:

Wenn ein Spielstein gelegt wird, so wird dieser aus der Liste gelöscht und wenn er wieder in die Spielsteinauswahl zurückgelegt wird, wird dieser wieder der Liste angefügt.

Hierbei bietet sich eine Arraylist an, da diese Dynamisch sich an neue Größen anpasst.

Hierdurch befindet sich ein Spielstein entweder im Spielfeld oder in der Spielsteinauswahl.

Die Reihenfolge wirkt hier erstmal durcheinander, jedoch kann dies ganz einfach gelöst werden.

Es kann einfach mithilfe einer Schleife welche solange läuft wie es Spielsteingesichter gibt, jeder Ordinale Wert der Spielsteingesichter durchlaufen werden.

Die Tiles Klasse hat eine Methode welche den zu dem aktuellen Schleifenindex passenden Spielstein (Schleifenindex und Ordinaler Wert des Spielsteingesichts passen) zurückgibt oder null falls sich dieser nicht in der Liste befindet.

Vorteil: ein Spielstein liegt entweder in der Spielsteinauswahl oder im Spielfeld (Konsistenz) und die Reihenfolge der Spielsteine bleibt weiterhin nutzbar.

Nachteil: erhöhter Ressourcenverbrauch, da pro gesuchtem Spielstein durch Ordinalen Index, jeder Spielstein der nicht gelegt ist durchlaufen werden muss.

Realisationsbeschreibung:

Die Wege die Spielsteine sobald sie gelegt sind aus der Spielsteinauswahl zu entfernen oder sie sobald die Spielsteine gelegt werden mit null zu in der Spielsteinauswahl zu ersetzen, sind am saubersten.

Es wurde sich für die Variante ohne das speichern mit null entschieden, da dies am saubersten wirkte, jedoch wird nachträglich festgestellt, dass die Variante mit null vom Rechenaufwand eindeutig effizienter ist und der Speicherverbrauch durch das Ersetzen gegenüber dem Löschen nur marginal höher ist.

Die gewählte Variante nutzt wie schon beschrieben eine Arraylist.

Wird ein Spielstein gelegt, so wird dieser aus der Arraylist gelöscht und wird dieser wieder zurück in die Spielsteinauswahl gelegt, so wird er wieder der Arraylist zugeführt.

Die Reihenfolge, welche z.B. für den Lösungs-Algorithmus essentiell ist wird durch eine Methode der Spielsteinauswahl Klasse gewährleistet.

Hierbei wird der Methode der Ordinale Index des Spielsteingesichts übergeben und die Methode läuft mithilfe einer Schleife jeden Spielstein durch und vergleicht von diesem den Ordinalen Index des Spielsteingesichts mit dem übergebenen Index.

Wurde einer gefunden, so wird dieser zurückgegeben, wenn nicht dann null.

2.5. Speichern der freien Felder eines Spielfelds für den Algorithmus zum Lösen des Spielfelds

Problemanalyse:

Laut Aufgabenstellung, soll es dem Spieler möglich sein, automatisch ein Spielfeld zu lösen um zu schauen, ob dies lösbar ist und falls gewünscht einen Spielstein als Hilfe automatisch im Spielfeld zu platzieren.

Hierzu wurde ein Algorithmus (Programmiererhandbuch 3.1) entwickelt, welcher eine Kopie des Spielfelds automatisch löst.

Hierbei werden immer wieder die freien Felder des Original Spielfelds auf und zurückgelaufen. Um dies zu tun, muss man wissen, wo das nächste freie Feld ist.

Realisationsanalyse:

Die freien Felder des Original Spielfelds, werden gespeichert:

Die freien Felder des Original Spielfeld werden alle abgelaufen.

Falls ein Feld des inneren Spielfelds leer ist, wird die Position von diesem in einer Liste vom Typ Position welche eine Koordinate darstellt gespeichert.

Vorteil: der Lösungs-Algorithmus kann einfach diese Felder vor und zurückgehen ohne das Spielfeld immer wieder nach dem nächsten oder vorherigen freien Feld von der aktuellen Position aus zu suchen (sollte Ressourceneffizienter in Hinsicht auf Prozessor sein).

Nachteil: es gibt einen höheren Ressourcenverbrauch bezüglich des Arbeitsspeichers welcher jedoch marginal ist.

Das nächste oder vorherige freie Feld wird immer wieder neu gesucht:

Ab der aktuellen Position im Spielfeld, werden alle Felder rückwärts oder vorwärts je nachdem was gewünscht ist abgelaufen, und das nächste freie Feld zurückgegeben.

Vorteil: nicht der marginale Arbeitsspeicherverbrauch.

Nachteil: höherer Ressourcenverbrauch in Hinsicht auf den Prozessor.

Realisationsbeschreibung:

Es wurden beide Alternativen getestet und gegen die Erwartungen, war die zweite, welche nicht die freien Felder speichert in den Tests schneller.

Somit muss jedes nächste Feld durch ablaufen aller Spielfeld Felder des Original Spielfelds ab der aktuellen Position geschehen und jedes vorherige Feld durch ablaufen aller vorherigen Felder ab der aktuellen Position.

3. Algorithmen

3.1 Algorithmus zum Lösen des Spielfelds

Das Spielfeld soll auf Lösbarkeit geprüft werden und es sollen Hinweise gelegt werden können. Hierfür muss das Spielfeld durch einen Algorithmus gelöst werden.

Entschieden habe ich mich für einen Brute Force Algorithmus mit Backtracking.

Die Vorgehensweise ist folgende:

1.
Zu Anfang wird vom originalen Spielfeld und den originalen Spielsteinen eine Kopie erstellt, damit das original Spielfeld als Vorlage der freien Löcher dient und das original Spielfeld nicht verändert wird.
2.
Nun läuft eine Hauptschleife solange, bis das Spielfeld gelöst wurde, oder bis das Spielfeld nicht gelöst wurde und jede Kombination ausprobiert wurde. Die Schleife umfasst die Punkt 3, 4, 5.
3.
Es wird nach dem ersten freien Feld im originalen Spielfeld gesucht, beginnend bei Position (1,1). Wurde eins gefunden, wird sich von diesem die Position gemerkt.
4.
Es wird ein passender, noch nicht gelegter Spielstein für die bestimmte Position gesucht. (Wann ein Spielstein passt, wird im Benutzerhandbuch Abschnitt 3.1 beschrieben)
Hierfür werden ab dem aktuell auf dem Feld liegenden Spielstein und der aktuellen Rotation des Spielsteins alle Spielsteine in allen Rotation ausprobiert.

+: Wird ein passender Spielstein gefunden, wird dieser gelegt und es geht wieder zu Schritt 3 im nächsten Schleifendurchlauf.

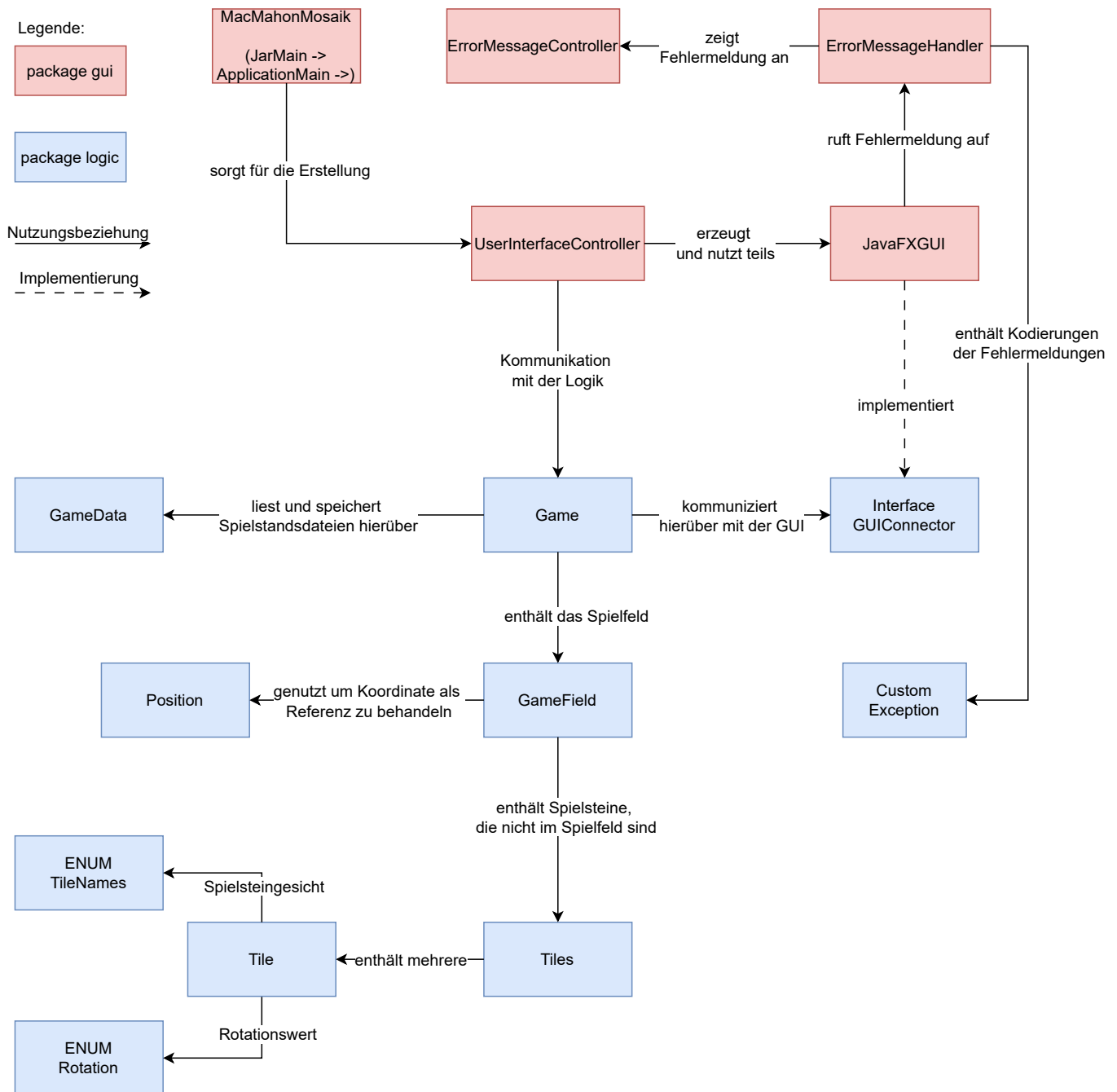
-: Wird kein passender Spielstein gefunden, wird kein Spielstein gelegt und es geht zu Schritt 5.
5.
Da kein passender Spielstein gefunden wurde, setzt nun das Backtracking ein.
Es wird nach dem vorherigen freien Feld im Originalfeld gesucht.

+: Gibt es ein vorheriges freies Feld im Originalfeld, so wird zu diesem gegangen und der aktuell dort liegende Spielstein im Kopie Spielfeld entfernt und im nächsten Schleifendurchlauf bei Schritt 4 gestartet, da man ja jetzt schon das nächste Feld gefunden hat, welches gefüllt werden soll.

-: Gibt es kein vorheriges freies Feld im Originalfeld, so ist der Algorithmus fertig und das Spielfeld ist nicht lösbar.

4. Programmorganisationsplan

4.1 Graphischer Programmorganisationsplan



Der Programmorganisationsplan wurde mithilfe von draw.io erstellt.

4.2 Programmorganisationsplan Erklärung

Das Programm ist in zwei Packages namens logic und gui unterteilt.

Das gui Package enthält alles, was für die graphische Darstellung des Spiels und für die Nutzerinteraktionen notwendig ist, sowie zusätzlich die Fehlerdialoge.

Das logic Package enthält alle nichtgraphischen Komponenten, welche zum Durchführen des Spiels notwendig sind.

Gestartet, wird das Programm in der JarMain Klasse, welche die ApplicationMain Klasse aufruft (beide zusammen im Diagramm unter MacMahonMosaik zusammengefasst).

Die ApplicationMain Klasse lädt das FXML Hauptfenster und instanziiert den dazugehörigen UserInterfaceController.

Der UserInterfaceController enthält eine Nutzlast des GUIConnector Interface, welches durch die JavaFXGUI Klasse implementiert wird und dann noch eine Nutzlast der Game Klasse.

Die vom UserInterfaceController erzeugte Game Klasse ist die Schnittstelle zwischen Nutzer und Logik und koordiniert alles.

Sie enthält eine Nutzlast der selbst erzeugten GameField Klasse und eine Nutzlast des vom UserInterfaceController erzeugten GUIConnectors.

Die GameField Klasse stellt ein Spielfeld dar, bestehend aus einer Nutzlast eines zweidimensionalen Arrays welches diese erzeugt vom Typ Tile.

Zusätzlich enthält die GameField Klasse eine selbst erzeugte Nutzlast des Typs Tiles, welche alle nicht gelegten Spielsteine der Klasse Tile enthält.

Die von GameField genutzte Klasse Tiles besteht aus mehreren Spielsteinen des Typs Tile.

Ein Tile stellt einen einzigen Spielstein dar und hat somit zwei Nutzlasten.

Zum Einen das Spielsteingefüge vom Enum Typ TileNames, welches vorgibt, wie der Stein aussieht und zum Anderen die Rotation eines Spielsteins vom Enum Typ Rotation.

Noch nicht beschrieben wurden die Klassen Position, CustomException, ErrorMessageHandler sowie ErrorMessageController.

Die Klasse Position enthält zwei Nutzlasten des Typs int und soll Koordinaten darstellen. Genutzt wird sie von der GameField Klasse, damit eine Methode zwei Werte gleichzeitig „zurückgeben“ kann, die in dieser Klasse gespeichert sind.

Die CustomException Klasse enthält die Codes der möglichen Fehlermeldungen. Diese werden von Klassen des logic und gui Package genutzt.

Die ErrorMessageHandler Klasse enthält die zu den CustomException gehörenden Fehlermeldungstexte und ruft bei Bedarf mit diesen Dialogen ein Fenster auf, welches diese anzeigt.

Bei dem Controller der Fenster welches die Dialoge anzeigt, handelt es sich um den ErrorMessageController.

Ausgelöst werden kann dieser Ablauf durch das GUIConnector Interface.

5. Dateien

Das Programm verwendet JSON Dateien zur Speicherung von Spielständen.
Die Struktur und Anforderungen an diese Dateien sind im Benutzerhandbuch, Abschnitt 5 beschrieben.

6. Programmtests

Testfall	Ergebnis
Programm wird gestartet	Wie zu erwarten, wird das Default Spielfeld der Größe 4x4 (inklusive Rand) mit gefärbten Rand mit leeren inneren Feldern geladen. Die Spielsteinauswahl enthält alle 24 Spielsteine in korrekter Reihenfolge. Der Editor-Mode ist deaktiviert.
Drehen eines Spielsteins in der Spielsteinauswahl	Der Spielstein dreht sich wie erwartet in 90° Schritten.
Drehen eines Spielsteins auf dem Spielfeld	Der Spielstein dreht sich wie erwartet in 90° Schritten.
Spielstein bewegen: Spielsteinauswahl -> Spielfeld	1. Freies Feld: Spielstein wird korrekt verschoben. 2. Belegtes Feld: Spielstein bleibt an seiner alten Position.
Spielstein bewegen: Spielsteinauswahl -> Spielsteinauswahl	Der Spielstein bleibt an seiner alten Position.
Spielstein bewegen: Spielfeld -> Spielsteinauswahl	Spielstein wird an die korrekte Position in der Spielsteinauswahl gesetzt.
Spielstein bewegen: Spielfeld -> Spielfeld	1. Freies Feld: Spielstein wird korrekt verschoben. 2. Belegtes Feld: Spielstein bleibt an seiner alten Position.
Lochstein im Spielfeld bewegen	1. Editor-Mode aktiv und freies Feld: Lochstein wird verschoben. 2. Editor-Mode aktiv und belegtes Feld: Lochstein wird nicht bewegt. 3. Editor-Mode nicht aktiv: Lochstein kann gar nicht erst bewegt werden.
Platzieren eines Spielsteins außerhalb des inneren Spielfelds sowie der Spielsteinauswahl	Der Spielstein geht auf sein Feld zurück, wo dieser sich vorher befand, da er nur im inneren Spielfeld oder der Spielsteinauswahl platziert werden kann.
Platzieren eines Lochsteins außerhalb des Spielfelds	Der Lochstein wird zurück auf seinen Platz gesetzt
Menü -> Spiel -> Spiel Neustarten; Editor-Mode inaktiv	Spielsteine im Spielfeld wandern zurück in die Spielsteinauswahl, Lochsteine bleiben unverändert sowie die Farben der Randfelder auch.

Menü -> Spiel -> Spiel Neustarten; Editor-Mode aktiv	Rand vollständig gefärbt: Fehlermeldung 504 (EditorMode muss beendet werden) wird korrekt angezeigt Rand unvollständig gefärbt: Fehlermeldung 503 (Rand ist unvollständig) wird korrekt angezeigt
Menü -> Spiel -> Spiel beenden; Editor-Mode inaktiv	1. Spiel ist aktiv: Spielfeld und Spielsteinauswahl wird deaktiviert und unscharf gemacht. 2. Spiel ist inaktiv: Spielfeld und Spielsteinauswahl bleiben deaktiviert und unscharf.
Menü -> Spiel -> Spiel beenden; Editor-Mode aktiv	Fehlermeldung 504 (EditorMode muss beendet werden) wird korrekt angezeigt
Menü -> Spiel -> Spiel speichern; Editor-Mode egal	Dateiauswahlfenster öffnet sich. Nachdem ein Speicherort angegeben wurde sowie ein Dateiname, den der Spielstand tragen soll, wird die Datei gespeichert. Die gespeicherte Datei entspricht allen Anforderungen einer Spielstandsdatei (nach Benutzerhandbuch Abschnitt 5)
Menü -> Spiel -> Spiel laden	Dateiauswahlfenster öffnet sich
Menü -> Spiel -> Spiel laden; Editor-Mode Egal, gültige Datei (valid.json)	Spielstand wird korrekt geladen und Spiel gestartet. Falls Editor-Mode aktiv war, wird dieser deaktiviert.
Menü -> Spiel -> Spiel laden; Spielsteinfenster vorzeitig schließen	Fehlermeldung 201 (kein Spielstand geladen) wird korrekt angezeigt, das Standardspiel wird wieder geladen.
Menü -> Spiel -> Spiel laden; JSON Datei hat falschen Schlüssel (invalid_wrongProperty.json)	Fehlermeldung 302 (Datei hat keinen „field“ Schlüssel) wird korrekt angezeigt, das Standardspiel wird wieder geladen.
Menü -> Spiel -> Spiel laden; JSON Datei hat beim Schlüssel kein Array (invalid_noJsonArray.json)	Fehlermeldung 303 (Datei hat beim „field“ Schlüssel den falschen Typ) wird korrekt angezeigt, das Standardspiel wird wieder geladen.
Menü -> Spiel -> Spiel laden; Datei ist leer (invalid_empty.json)	Fehlermeldung 305 (Datei ist leer) wird korrekt angezeigt, das Standardspiel wird wieder geladen.
Menü -> Spiel -> Spiel laden; Spielfeld der Datei ist zu klein bezüglich Breite (invalid_tooSmallWidth.json)	Fehlermeldung 304 (Spielfeld hat falsche Größe) wird korrekt angezeigt, das Standardspiel wird wieder geladen.
Menü -> Spiel -> Spiel laden; Spielfeld der Datei ist zu klein bezüglich Höhe (invalid_tooSmallHeight.json)	Fehlermeldung 304 Spielfeld hat falsche Größe) wird korrekt angezeigt, das Standardspiel wird wieder geladen.
Menü -> Spiel -> Spiel laden; Spielfeld der Datei ist zu groß bezüglich Breite (invalid_tooBigWidth.json)	Fehlermeldung 304 Spielfeld hat falsche Größe) wird korrekt angezeigt, das Standardspiel wird wieder geladen.

Menü -> Spiel -> Spiel laden; Spielfeld der Datei ist zu groß bezüglich Höhe (invalid_tooBigHeight.json)	Fehlermeldung 304 Spielfeld hat falsche Größe) wird korrekt angezeigt, das Standardspiel wird wieder geladen.
Menü -> Spiel -> Spiel laden; Spielfeld der Datei hat nicht vollständig gesetzten Rand (8x7_BorderInvalid_3TilesLaid.json)	Editor-Mode wird aktiviert und Fehlermeldung 503 (Rand nicht vollständig gefärbt) wird korrekt angezeigt. Anmerkung: Hier hat das Programm einen Fehler, der EditorMode wird zwar aktiviert und die Spielsteinauswahl deaktiviert, doch falls der Spielstand Spielsteine im Spielfeld hat, werden diese trotz aktivierten EditorMode nicht entfernt.
Menü -> Lösbarkeit -> Lösbarkeit Prüfen; Editor-Mode egal	Spielfeld <= 18 innere Felder: Spiel ist lösbar: Mitteilung 10, Spiel ist nicht lösbar: Mitteilung 11 Spielfeld > 18 innere Felder: Mitteilung 31 (Lösbarkeit und Lösungstipps stehen nicht zur Verfügung)
Menü -> Lösbarkeit -> Lösungstipp; Editor-Mode aktiv	Spielfeld <= 18 innere Felder: Fehler 504 (Editor muss deaktiviert sein) Spielfeld > 18 innere Felder: Mitteilung 31 (Lösbarkeit und Lösungstipps stehen nicht zur Verfügung)
Menü -> Lösbarkeit -> Lösungstipp; Editor-Mode inaktiv	Spielfeld <= 18 innere Felder: Spiel ist lösbar: Mitteilung 21 (kann keinen Spielstein legen, da Spielfeld nicht lösbar) Spiel ist nicht lösbar: Mitteilung 11 Spielfeld > 18 innere Felder: Mitteilung 31 (Lösbarkeit und Lösungstipps stehen nicht zur Verfügung)
Menü -> Editor -> Bearbeitungsmodus	Spiel ist inaktiv: Spielfeld wird aktiviert, Spielsteine werden zurück in die Spielsteinauswahl gelegt. Spiel ist aktiv: Spielsteinauswahl wird deaktiviert, Spielsteine werden zurück in die Spielsteinauswahl gelegt.

7. Bekannte Fehler des Programms:

Commitf: 6df0d46bab18acc9d80fa4d0351ee10cd2034ac

1. Wird ein Spiel in Editor-Modus mit nicht vollständig gesetztem Rand geladen, werden die Spielsteine falls welche im Feld waren nicht zurückgesetzt.